

实验三：动态扫描显示

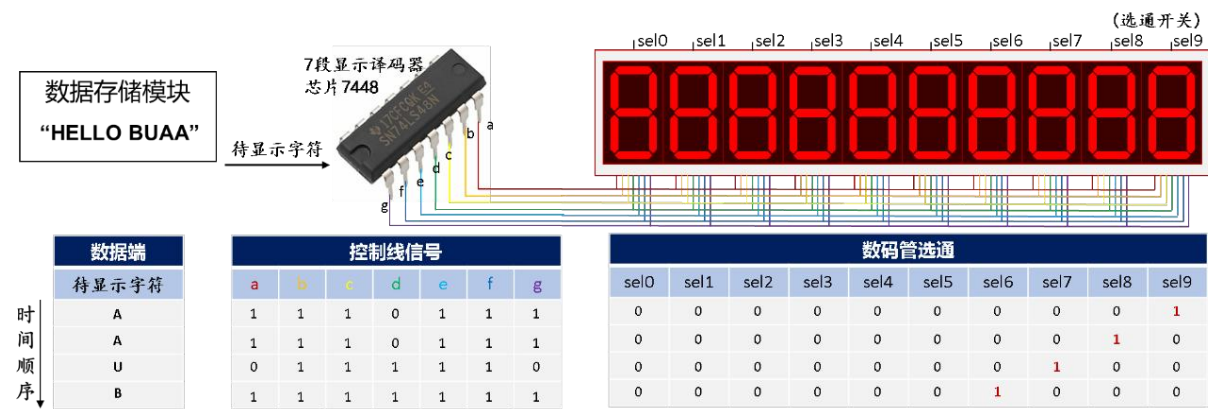
实验日期：2025.11.7
姓名： 何宗霖
学号：23374275
班级：231516
任课教师：李家军

一、实验目的

- 1. 掌握按键消抖的原理及实现
- 2. 掌握数码管动态扫描原理及实现

二、实验原理

- 1. 动态扫描原理：分时复用控制线
为解决控制线过多的问题，采用多个像素共用同一根控制线；
为每个数码管显示内容一样，增加选通开关，每一时刻只有一个数码管显示。



- 2. 视觉暂留效应：光对视网膜所产生的视觉在光停止作用后，仍保留一段时间现象，如果在视觉暂留效应消失前，数码管再一次被选通，即可实现连续显示效果。
- 3. 扫描速度（刷新率）指标，即每秒数码管被选通的次数；
- 4. 扫描方式即同时点亮的行数与整个区域行数的比例。
- 5. 动态扫描优缺点

优缺点分析	静态扫描	动态扫描
亮度	高	一般
显示效果	好	一般
成本	高	低
功耗	高	低
应用场景	户外高清	室内场景

6. 动态扫描驱动电路

产生数码管选通信号：任意时刻的控制信号中只有一位为“1”，且“1”的切换必须满足一定的时间要求。

- 扫描频率：1KHz
(隔 0.001 秒切换切换数码管)
- 需要时钟分频电路
- 顺序脉冲发生器
- 环形移位寄存器
- 计数器+译码器

数码管选通									
sel0	sel1	sel2	sel3	sel4	sel5	sel6	sel7	sel8	sel9
0	0	0	0	0	0	0	0	0	1
0	0	0	0	0	0	0	0	1	0
0	0	0	0	0	0	0	1	0	0
0	0	0	0	0	1	0	0	0	0
0	0	0	0	1	0	0	0	0	0
0	0	1	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	1	0
0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	1	0	0	0	0
0	0	0	1	0	0	0	0	0	0
0	0	1	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	1	0	0
0	0	0	0	0	1	0	0	0	0
0	0	0	1	0	0	0	0	0	0
0	0	1	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0

三、实验内容

1. (必做) 利用数码管显示 “HI BUAA”
2. (必做) 让 “HI BUAA” 在数码管上滚动播放

四、硬件资源

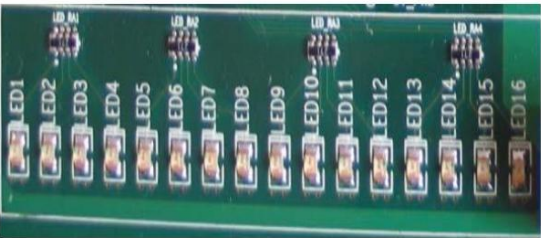
1. 实验用 FPGA 芯片:在本实验中, 使用了 Altera (Intel) 公司 Cyclone IV 系列 FPGA 芯片, 型号为 EP4CE10F17C8N, 有近 10k 逻辑资源, 速度等级为 8, FBGA 封装, 256 个 IO 引脚。

资源	EP4CE6	EP4CE10	EP4CE15	EP4CE22	EP4CE30	EP4CE40	EP4CE55	EP4CE75	EP4CE115
逻辑单元 (LE)	6, 272	10, 320	15, 408	22, 320	28, 848	39, 600	55, 856	75, 408	114, 480
嵌入式存储器 (Kbits)	270	414	504	594	594	1, 134	2, 340	2, 745	3, 888
嵌入式 18 × 18 乘法器	15	23	56	66	66	116	154	200	266
通用 PLL	2	2	4	4	4	4	4	4	4
全局时钟网络	10	10	20	20	20	20	20	20	20
用户 I/O 块	8	8	8	8	8	8	8	8	8
最大用户 I/O (注释 1)	179	179	343	153	532	532	374	426	528

【注】：EP4CE10F17C8N 为 PPT 上所写, 实际 Quartus 用的是 EP4CE115F23C7 型号的芯片, PPT 中并未作出介绍

2.LED 指示灯:

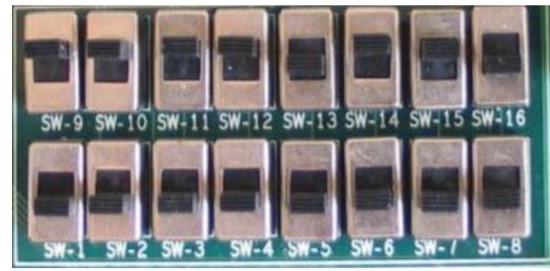
开发平台上有 2x8 共 16 个 LED 指示灯, 便于对用户的操作给出提示, 使用户得到相应的反馈信息, 也可验证设计的正确性。注意为低电平时亮。



LED1	0V_D5	由开关 VLP0 选择	CON1. 2	C10
LED2	0V_D6	由开关 VLP0 选择	CON1. 3	E9
LED3	0V_D7	由开关 VLP0 选择	CON1. 4	D10
LED4	0V_D8	由开关 VLP0 选择	CON1. 5	V13
LED5	0V_D9	由开关 VLP0 选择	CON1. 6	V14
LED6	VSYNC	由开关 VLP0 选择	CON1. 7	Y17
LED7	PCLK	由开关 VLP0 选择	CON1. 8	W17
LED8	0V_RES	由开关 VLP0 选择	CON1. 9	W19

3.拨码开关：

开发平台上有二组 16 只拨码开关输入，开关拨置上为逻辑“1”电平，开关拨置下为逻辑“0”电平。



SW1			CON1. 33	M16
SW2			CON1. 34	R19
SW3			CON1. 54	AA14
SW4			CON1. 61	AA1
SW5			CON2. 62	A3
SW6			CON2. 61	B4
SW7			CON2. 60	C4
SW8			CON2. 59	E6

4.对应关系：

在本次基础实验中，管脚分配如下表

set_location_assignment	PIN_T1	clk_sys
set_location_assignment	PIN_M16	roll
set_location_assignment	PIN_P20	seg[0]
set_location_assignment	PIN_R20	seg[1]
set_location_assignment	PIN_R17	seg[2]
set_location_assignment	PIN_P22	seg[3]
set_location_assignment	PIN_R21	seg[4]
set_location_assignment	PIN_T17	seg[5]
set_location_assignment	PIN_R22	seg[6]
set_location_assignment	PIN_U21	seg[7]
set_location_assignment	PIN_W21	sel[7]
set_location_assignment	PIN_AA20	sel[6]
set_location_assignment	PIN_AA19	sel[5]
set_location_assignment	PIN_AB20	sel[4]
set_location_assignment	PIN_AA17	sel[3]
set_location_assignment	PIN_AB16	sel[2]
set_location_assignment	PIN_AB17	sel[1]
set_location_assignment	PIN_T18	sel[0]
set_location_assignment	PIN_AB14	resetn_b

4. 外设

➤ 数码管



seg控制一个数码管哪一段是否发光

设计端口	引脚分配	开发平台模块
Seg[7]	U21	8xSEG LA
Seg[6]	R22	8xSEG LB
Seg[5]	T17	8xSEG LC
Seg[4]	R21	8xSEG LD
Seg[3]	P22	8xSEG LE
Seg[2]	R17	8xSEG LF
Seg[1]	R20	8xSEG LG
Seg[0] (高使能)	P20	8xSEG LH
Sel[1]	AB17	8xSEG DS2
Sel[0] (低使能)	T18	8xSEG DS1

数码管段选

Sel为数码管使能端

控制某一个数码管是否被使能

sel[7]	B12	W21	8xSEG DS8
sel[6]	K20	AA20	8xSEG DS7
sel[5]	J18	AA19	8xSEG DS6
sel[4]	H11	AB20	8xSEG DS5
sel[3]	L7	AA17	8xSEG DS4
sel[2]	L8	AB16	8xSEG DS3
sel[1]	K9	AB17	8xSEG DS2
sel[0]	H6	T18	8xSEG DS1

数码管位选

5. 外设

点动开关

- KSI拨下时F7-F10可用
- 电动开关未按下时输出高电平，按下时输出低电平



实物图片

F1			CON1.10	AB14
F2			CON1.11	W20
F3			CON1.12	AA15
F4			CON1.13	AB15
F5			CON1.14	T15
F6			CON1.15	U20
F7	SMBUS_SDA	由开关 KSI 选择	CON1.16	AA18
F8	SMBUS_SCL	由开关 KSI 选择	CON1.17	AA16
F9	I2C_SCL	由开关 KSI 选择	CON1.18	AB18
F10	I2C_SDA	由开关 KSI 选择	CON1.19	AB19

五、实验过程与结果

1. 利用数码管显示“HI BUAA”+让“HI BUAA”在数码管上滚动播放。

实验核心代码：

```
// 动态扫描 8 个数码管，显示 "HI BUAA"（带空格填充），roll=1 时每 1s 左滚动一个字符
module ScanDisplay #(
    parameter integer CLOCK_FREQ = 25_000_000 // sys clock freq, default 25MHz
    //定义这个在哪里用
)
(
    input          clk_sys,
    input          resetn_b, // low active reset
    input          roll,    // 开关: 1 = 滚动每1s, 0 = 停止
    output [7:0] seg,      // {dp,g,f,e,d,c,b,a} 高有效，这个是七段码，dp表示小数点
    output [7:0] sel       // 低有效，哪一位为0表示选中对应数码管
);

// -----
// 参数 / 计数器，用于刷新和 1s 滚动定时
// -----
localparam REFRESH_COUNTER_WIDTH = 16; // 决定扫描速率
localparam SCROLL_COUNTER_WIDTH = 32; // 决定滚动速率
reg [REFRESH_COUNTER_WIDTH-1:0] refresh_cnt;
reg [SCROLL_COUNTER_WIDTH-1:0] scroll_cnt; //滚动计数
localparam [SCROLL_COUNTER_WIDTH-1:0] SCROLL_TARGET = CLOCK_FREQ - 1;
//这俩通过计数实现分频

// 使用 refresh_cnt 的高 3 bit 做index索引（固定0..7循环）
wire [2:0] index = refresh_cnt[REFRESH_COUNTER_WIDTH-1 -: 3];
//这句不用写到always里面吗

reg [2:0] scroll_pos; // 滚动位置偏移
reg [2:0] digit_idx; // 选通数码管索引
wire [2:0] content_idx; //显示字符的索引

// -----
// 预定义 7 段码（高有效，bit7=dp）
// 常用段码（参考常见编码：0x3F,0x06,...），并对字母作近似表示
// -----
//在C、C++、py等0x是十六进制，在verilog里h表示十六进制
localparam [7:0] SEG_SPACE = 8'h00; // 00000000
localparam [7:0] SEG_A = 8'hEE; // A 11101110
localparam [7:0] SEG_B = 8'hFE; // B (像 B) 11111110
localparam [7:0] SEG_H = 8'h6E; // H (近似) 01101110
localparam [7:0] SEG_I = 8'h60; // I as '1' style 01100000
localparam [7:0] SEG_U = 8'h7C; // U 01111100
```



```

always @(posedge clk_sys or negedge resetn) begin
    if(~resetn) begin
        refresh_cnt<=0;
        scroll_cnt<=0;
        scroll_pos<=0;
        digit_idx<=0;
        //没有处理content_idx, 因为content_idx=digit_idx + scroll_pos
    end
    else begin
        // refresh_cnt始终+1, 提供稳定的index 0--7循环
        refresh_cnt <= refresh_cnt+1;

        //选通数码管根据index切换
        digit_idx <= index;

        // 若不滚动显示, scroll_pos始终为0
        if(~roll) begin
            // 补充以下代码
            scroll_cnt<=scroll_cnt;
            scroll_pos<=scroll_pos;
        end
        // 若滚动显示, scroll_pos每隔一定时间增1
        else begin
            // 若滚动计数满, 则触发一次滚动
            if (scroll_cnt == SCROLL_TARGET) begin
                //不是计数满了, [SCROLL_COUNTER_WIDTH-1:0] scroll_cnt还没满
                //而是1s滚动一次, 计数的频率是CLOCK_FREQ, 时间为1s
                scroll_cnt <= 0; //清零
                scroll_pos <= scroll_pos + 1'd1;
            end else begin
                scroll_cnt <= scroll_cnt + 1;
                // scroll_pos 保持不变!
            end
        end
    end
end
end
end

```

```

reg [7:0] cur_pattern;
// 计算当前显示字符索引: 窗口起始 scroll_pos, 加上 digit_idx (向左滚动意味着窗口起始向右移动)
assign content_idx = digit_idx + scroll_pos; //向左滚动, 因为+1, H变为I
//assign content_idx = digit_idx - scroll_pos;实现向右滚动
always @(*) begin
    case(content_idx)
        3'd0: cur_pattern=SEG_SPACE;
        3'd1: cur_pattern=SEG_H;
        3'd2: cur_pattern=SEG_I;
        3'd3: cur_pattern=SEG_SPACE;
        3'd4: cur_pattern=SEG_B;
        3'd5: cur_pattern=SEG_U;
        3'd6: cur_pattern=SEG_A;
        3'd7: cur_pattern=SEG_A;
        default: cur_pattern=SEG_SPACE;
    endcase
    // seg 直接等于当前显示字符的段码
    // 如果你想打开小数点, 可以设置 cur_pattern[7]=1
end

assign sel = ~(8'h80 >> digit_idx); //低电平有效 10000000
//digit_idx = 0 → sel = 8'b0111_1111 → 选中的是第 7 位 (sel[7]=0)
//digit_idx = 7 → sel = 8'b1111_1110 → 选中的是第 0 位 (sel[0]=0)
//从最左边到最右边依次亮
//相反的: assign sel = ~(8'h01 << digit_idx);
assign seg = cur_pattern; //高电平有效

```

```

//消抖模块
wire resetn;
debouncing debouncing_obj(clk_sys, resetn_b, resetn, );

```

```
endmodule
```

代码原理通俗解释：

- content_idx 控制背后亮的是什么
- 本来对应的是 hi buaa 的顺序
- 某一位亮了之后，对应位置的字母就出现了
- （指针先指向门，门开了，门的指针直指门后的数据）
- 有了偏移
- 指针先指向门，门开了，门的指针没指门后的数据，指了旁边门后的数据
- 又因为 content_idx = digit_idx + scroll_pos;
- 所以+1 的偏移，让 0 门指向 1 数据，H 打开是 I（左移）
- 所以如果要想实现右移，只需要 content_idx = digit_idx - scroll_pos

防抖模块：

```
module debouncing #(
    parameter bitwidth = 20,
    parameter delayT    = 250_000 // e.g. 对于 50MHz 时钟约 5ms 延迟
)(
    input wire clk_sys, // 系统时钟
    input wire key_b,   // 原始按键输入（低有效）
    output reg key,      // 去抖动后的按键信号（低有效），信号持续时间约为按下时间
    output key_pulse     // 去抖动后的按键信号（低有效），信号仅持续一个周期
);

//=====
// 内部寄存器
//=====
reg [bitwidth-1:0] counter = delayT; // 去抖计数器
reg prev_key; // 记录原来的key值，用于生成脉冲key_pulse

//=====
// 计数逻辑：
// 当检测到按键按下（key_b=0 且 key=1）时，启动计数。
// 按键保持低电平时计数器递增；
// 计数达到 delayT 时认为按键稳定。
//=====
always @(negedge clk_sys) begin
    if ((counter == delayT) && (key_b == 1'b0) && (key == 1'b1)) begin
        counter <= 0; // 检测到按下瞬间，启动计数
    end
    else if (counter < delayT) begin
        counter <= counter + 1'b1; // 继续计数
    end
    // 当 counter == delayT 时保持不变
end

always @(negedge clk_sys) begin
    if ((counter == delayT) && (key_b == 1'b0)) begin
        key <= 1'b0; // 稳定按下
    end
    else if (key_b == 1'b1) begin
        key <= 1'b1; // 立即恢复为高电平（释放）
    end
    prev_key <= key;
end

// 仅当key下降沿，key_pulse为0
assign key_pulse = ~(prev_key & (~key));

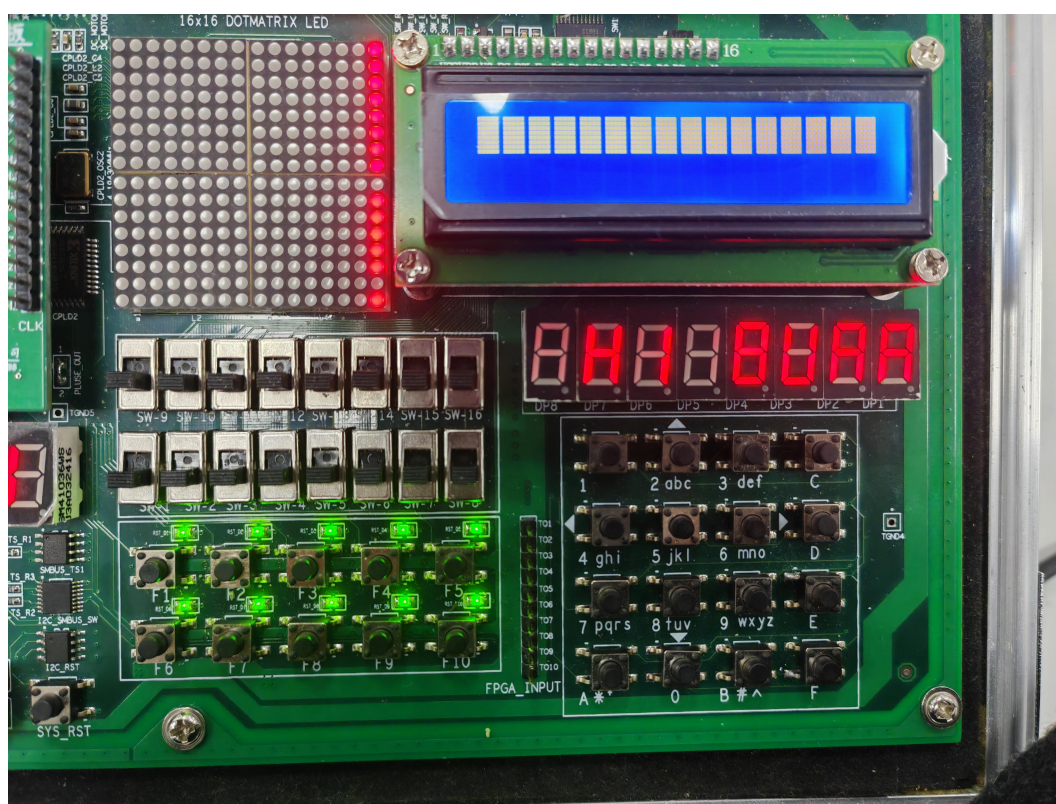
endmodule
```

管脚绑定情况界面：

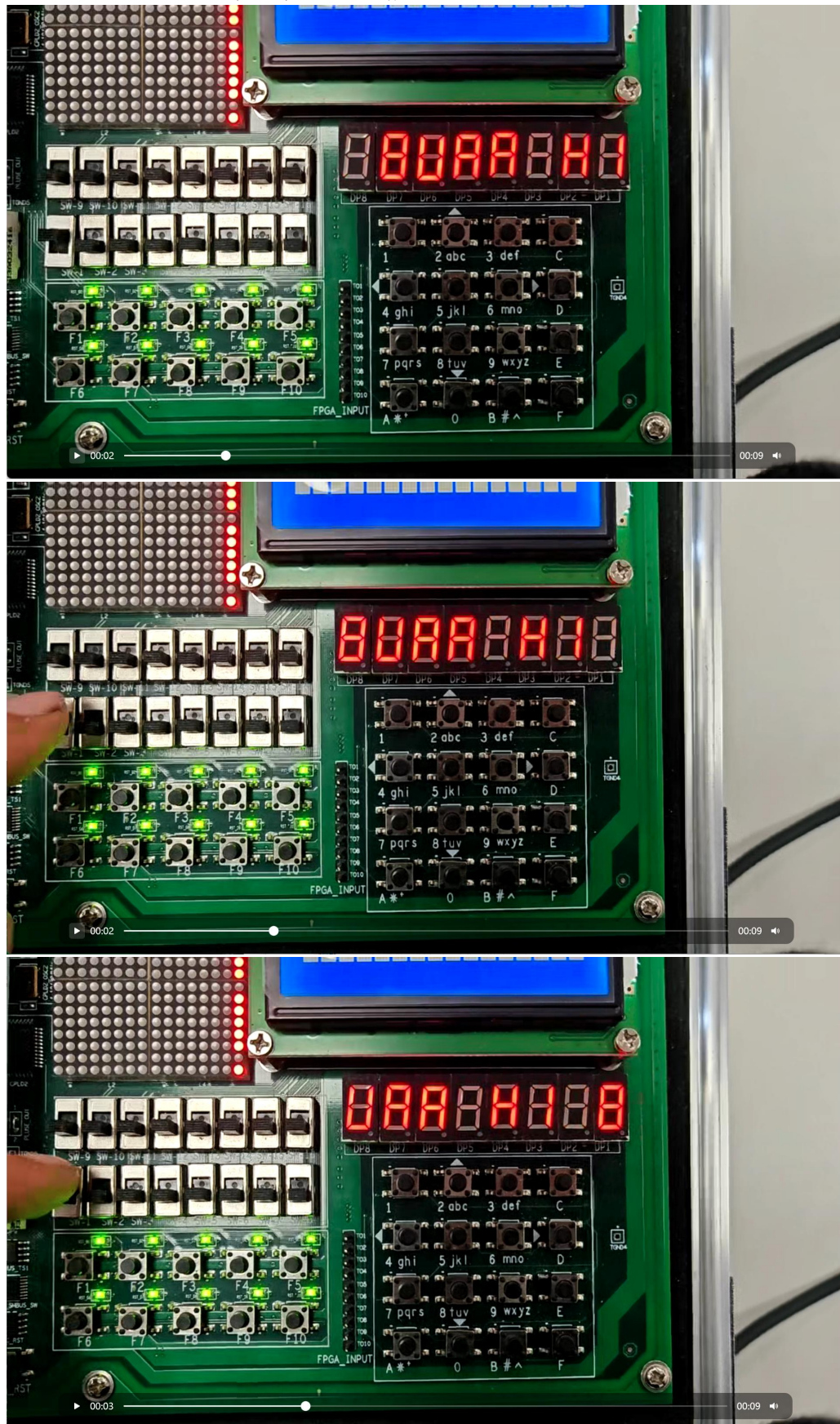
Node Name	Direction	Location	I/O Bank	VREF Group	Pin Location	I/O Standard	Reserved	Current Strength	Slew Rate	Differential Pair
clk_sys	Input	PIN_T1	2	B2_N0	PIN_T1	2.5 V ...fault		8mA (...ault)		
roll	Input	PIN_M16	5	B5_N2	PIN_M16	2.5 V ...fault		8mA (...ault)		
seq[0]	Output	PIN_P20	5	B5_N1	PIN_P20	2.5 V ...fault		8mA (...ault)	2 (default)	
seq[1]	Output	PIN_R20	5	B5_N1	PIN_R20	2.5 V ...fault		8mA (...ault)	2 (default)	
seq[2]	Output	PIN_R17	5	B5_N2	PIN_R17	2.5 V ...fault		8mA (...ault)	2 (default)	
seq[3]	Output	PIN_P22	5	B5_N1	PIN_P22	2.5 V ...fault		8mA (...ault)	2 (default)	
seq[4]	Output	PIN_R21	5	B5_N1	PIN_R21	2.5 V ...fault		8mA (...ault)	2 (default)	
seq[5]	Output	PIN_T17	5	B5_N2	PIN_T17	2.5 V ...fault		8mA (...ault)	2 (default)	
seq[6]	Output	PIN_R22	5	B5_N1	PIN_R22	2.5 V ...fault		8mA (...ault)	2 (default)	
seq[7]	Output	PIN_U21	5	B5_N1	PIN_U21	2.5 V ...fault		8mA (...ault)	2 (default)	
seq[7]	Output	PIN_W21	5	B5_N2	PIN_W21	2.5 V ...fault		8mA (...ault)	2 (default)	
sel[6]	Output	PIN_AA20	4	B4_N1	PIN_AA20	2.5 V ...fault		8mA (...ault)	2 (default)	
sel[5]	Output	PIN_AA19	4	B4_N1	PIN_AA19	2.5 V ...fault		8mA (...ault)	2 (default)	
sel[4]	Output	PIN_AB20	4	B4_N1	PIN_AB20	2.5 V ...fault		8mA (...ault)	2 (default)	
sel[3]	Output	PIN_AA17	4	B4_N1	PIN_AA17	2.5 V ...fault		8mA (...ault)	2 (default)	
sel[2]	Output	PIN_AB16	4	B4_N2	PIN_AB16	2.5 V ...fault		8mA (...ault)	2 (default)	
sel[1]	Output	PIN_AB17	4	B4_N1	PIN_AB17	2.5 V ...fault		8mA (...ault)	2 (default)	
sel[0]	Output	PIN_T18	5	B5_N2	PIN_T18	2.5 V ...fault		8mA (...ault)	2 (default)	
resetn_b	Input	PIN_AB14	4	B4_N2	PIN_AB14	2.5 V ...fault		8mA (...ault)		
<<new node>>										

最终结果：

1.静态显示 “HI BUAA”

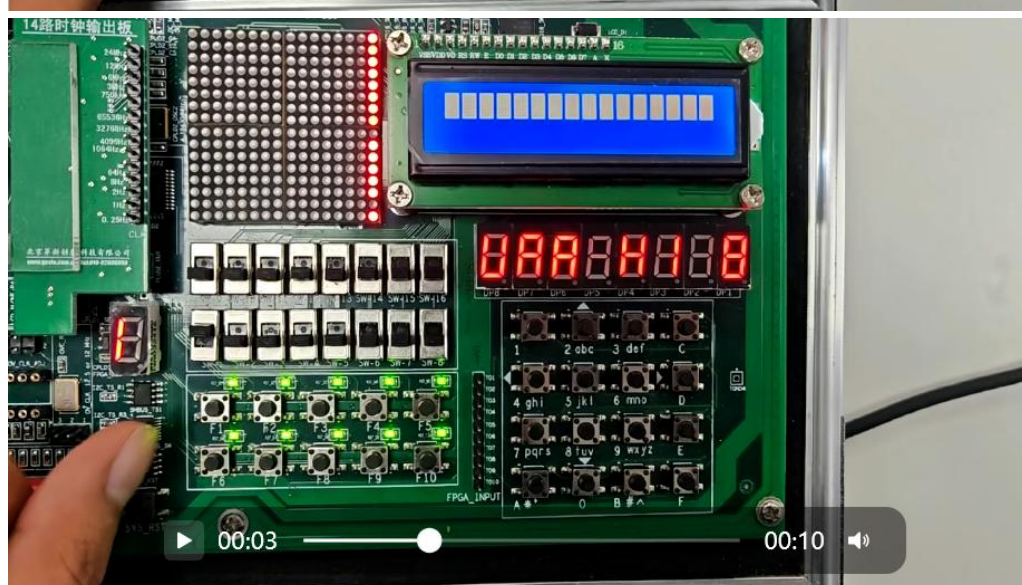
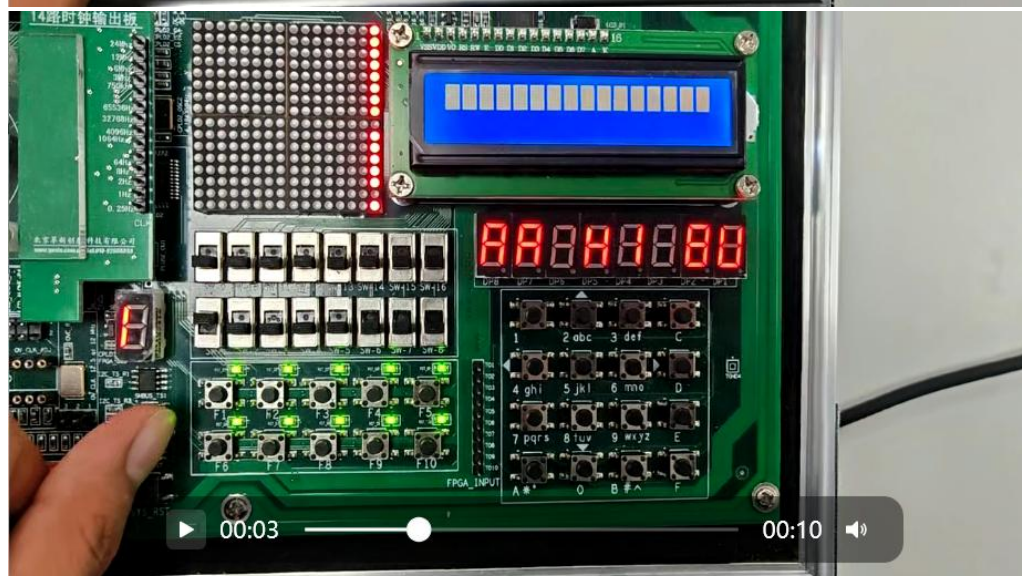


2. “HI BUAA” 在数码管上滚动播放



3.根据前文“通俗”理解：

令 $\text{content_idx} = \text{digit_idx} - \text{scroll_pos}$ ，实现右移



4.调整 scroll_cnt 实现加速

从 `if (scroll_cnt == SCROLL_TARGET) begin`

```
//不是计数满了, [SCROLL_COUNTER_WIDTH-1:0] scroll_cnt 还没满
```

```
//而是 1s 滚动一次, 计数的频率是 CLOCK_FREQ, 时间为 1s
```

```
scroll_cnt <= 0;//清零
```

```
scroll_pos <= scroll_pos + 1'd1;
```

```
end else begin
```

```
scroll_cnt <= scroll_cnt + 1;
```

```
// scroll_pos 保持不变!
```

```
end
```

改为 `if (scroll_cnt == SCROLL_TARGET/2) begin`

```
//不是计数满了, [SCROLL_COUNTER_WIDTH-1:0] scroll_cnt 还没满
```

```
//而是 1s 滚动一次, 计数的频率是 CLOCK_FREQ, 时间为 1s
```

```
scroll_cnt <= 0;//清零
```

```
scroll_pos <= scroll_pos + 1'd1;
```

```
end else begin
```

```
scroll_cnt <= scroll_cnt + 1;
```

```
// scroll_pos 保持不变!
```

```
end
```

以上 4 个实验结果详情均见视频

六、实验仿真波形

1. 利用数码管显示“HI BUAA”

编写 ScanDisplay_tb.v 代码

```
1  `timescale 1ns/1ns //照常写
2
3  //
4  module ScanDisplay_tb;
5      // 测试台输入输出信号定义
6      reg clk_sys;
7      reg resetn_b;
8      reg roll;
9      wire [7:0] sel; //数码管使能信号
10     wire [7:0] seg; //数码管段信号
11
12     // parameter CLOCK_FREQ = 25_000_000这个是实际器件，仿真不需要
13     // parameter ClockPeriod = 40; //如果取40ns就是对应CLOCK_FREQ
14     parameter ClockPeriod = 10;
15     always #(ClockPeriod/2) clk_sys = ~clk_sys;
16
17     initial begin
18         $dumpfile("ScanDisplay.vcd"); // 设置波形输出文件
19         $dumpvars(0, ScanDisplay_tb); //照常写
20
21         //初始化
22         clk_sys = 1'b0;
23         resetn_b = 1'b0;
24         roll = 1'b0;
25
26         #100
27         resetn_b = 1'b1;
28         roll=1;
29
30         // #5_000_000_000 resetn_b = 0;// 5000_000_000ns对应5s后复位结束
31         //太久了
32         #1_000_000_000 resetn_b = 0;// 1000_000_000ns对应1s后复位结束
33         $finish; //
34     end
35
36     ScanDisplay ScanDisplay_obj (.clk_sys(clk_sys), .resetn_b(resetn_b), .roll(roll), .seg(seg),.sel(sel));
37     //实例化
38
39 endmodule
```

1) 编译文件

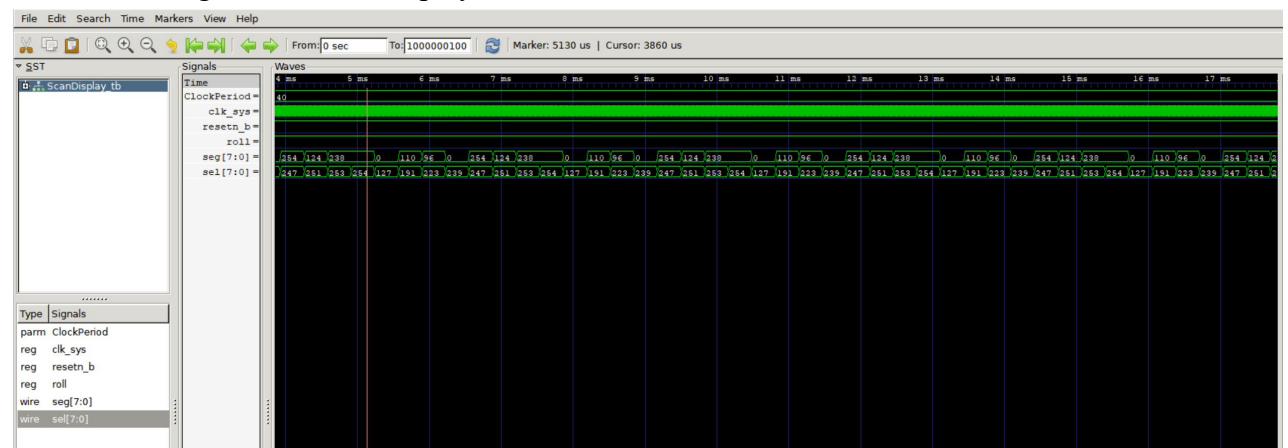
命令行: iverilog -o ScanDisplay.out ScanDisplay.v ScanDisplay_tb.v

2) 生成 .vcd 波形文件

命令行: vvp ScanDisplay.out

3) 观察波形

命令行: gtkwave ScanDisplay.vcd



2. 让“HI BUAA”在数码管上滚动播放

编写 ScanDisplay_tb.v 代码（只需使时间更久，观察到 scroll_pos 发生变化）

```
1  `timescale 1ns/1ns //照常写
2
3  //
4  module ScanDisplay_tb;
5      // 测试台输入输出信号定义
6      reg clk_sys;
7      reg resetn_b;
8      reg roll;
9      wire [7:0] sel; //数码管使能信号
10     wire [7:0] seg; //数码管段信号
11
12     // parameter CLOCK_FREQ = 25_000_000这个是实际器件，仿真不需要
13     // parameter ClockPeriod = 40; //如果取40ns就是对应CLOCK_FREQ
14     parameter ClockPeriod = 10;
15     always #(ClockPeriod/2) clk_sys = ~clk_sys;
16
17     initial begin
18         $dumpfile("ScanDisplay.vcd"); // 设置波形输出文件
19         $dumpvars(0, ScanDisplay_tb); //照常写
20
21         //初始化
22         clk_sys = 1'b0;
23         resetn_b = 1'b0;
24         roll = 1'b0;
25
26         #100
27         resetn_b = 1'b1;
28         roll=1;
29
30         // #5_000_000_000 resetn_b = 0; // 5000_000_000ns对应5s后复位结束
31         //太久了
32         #1_000_000_000 resetn_b = 0; // 1000_000_000ns对应1s后复位结束
33         $finish; //
34     end
35
36     ScanDisplay ScanDisplay_obj (.clk_sys(clk_sys), .resetn_b(resetn_b), .roll(roll), .seg(seg),.sel(sel));
37     //实例化
38
39 endmodule
```

1) 编译文件

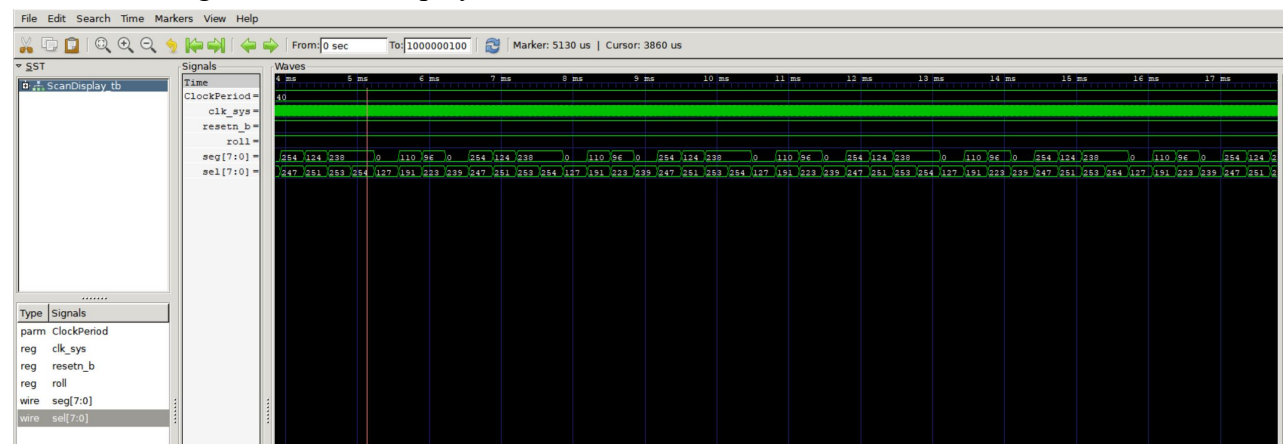
命令行: iverilog -o ScanDisplay.out ScanDisplay.v ScanDisplay_tb.v

2) 生成 .vcd 波形文件

命令行: vvp ScanDisplay.out

3) 观察波形

命令行: gtkwave ScanDisplay.vcd



七、附加实验仿真波形

无附加实验，两个实验均为必做实验。